

Numerical Analysis

Complete Study Guide

From Basics to Advanced Topics

Topics Covered:

Numerical Differentiation & Integration • Taylor Series • Simpson's Rule
Gaussian Quadrature • Optimization • Convex Sets & Functions
Gradient Descent • Eigenvalues & Eigenvectors • Lagrange Multipliers
Duality • Conjugate Functions • KKT Conditions
Sparse Matrix Computations • RL Applications • Reinforcement Learning

Based on Class Lectures: January 3 – February 24

Compiled for Exam Preparation

Contents

1	Introduction to Numerical Analysis	3
1.1	Why is Numerical Analysis Important?	3
1.2	Key Areas We Will Cover	3
2	Taylor Series & Numerical Differentiation	4
2.1	Taylor Series Expansion	4
2.2	Numerical Differentiation	4
2.2.1	Forward Difference Formula	4
2.2.2	Central Difference Formula	5
3	Numerical Integration	6
3.1	Trapezoidal Rule	6
3.2	Simpson's Rule	6
3.3	Comparison of Integration Methods	7
3.4	Gaussian Quadrature	7
4	Introduction to Optimization	9
4.1	Types of Optimization Problems	9
4.2	Standard Form of an Optimization Problem	9
5	Convex Sets & Convex Functions	11
5.1	Convex Sets	11
5.1.1	Important Types of Convex Sets	11
5.2	Convex Functions	12
5.3	Three Ways to Check Convexity	12
6	Linear Algebra for Optimization	14
6.1	Eigenvalues and Eigenvectors	14
6.2	Positive Semi-Definite (PSD) Matrices	14
6.3	Ellipsoids and Matrices	15
7	Gradient Descent	16
7.1	Descent Direction	16
7.2	Constrained Optimization & Indicator Functions	17
7.3	Subgradients	17
7.4	Norms	17
8	Lagrange Multipliers & the Lagrangian	18
8.1	The Lagrangian Function	18
8.2	Feasible Set	19
9	Duality in Optimization	20
9.1	The Dual Function	20
9.2	The Dual Problem	20
9.3	Weak and Strong Duality	21
9.4	Slater's Condition	21
10	Conjugate Functions	22
10.1	Connection to Duality	22
10.2	Wave-Particle Duality Analogy	22

11 KKT Conditions	24
12 Applications of Optimization	25
12.1 Low-Rank Matrix Factorization	25
12.2 Recommendation Systems	25
12.3 Singular Value Decomposition (SVD)	25
13 Sparse Matrix Computations & RL Applications	26
13.1 LU Factorization Recap	26
13.2 Sparse Matrices	26
13.2.1 Coordinate (COO) Storage Format	27
13.2.2 Compressed Sparse Row (CSR) Format	27
13.3 The Fill-in Problem	28
13.4 Formulating Sparse Matrix Reordering as an RL Game	28
13.5 Other RL Applications Discussed in Class	29
13.6 Key Insight: Learning at the Feature Level	30
14 Reinforcement Learning	31
14.1 Key Concepts	31
14.2 Exploration vs. Exploitation	31
14.3 Actor-Critic Methods	32
14.4 Markov Decision Process (MDP)	32
15 Complete Formula Sheet	33
16 Practice Exam Questions with Answers	35
17 Quick Revision Checklist	38

Introduction to Numerical Analysis

What is Numerical Analysis?

Numerical Analysis is the study of numerical methods for solving mathematical problems. It focuses on **approximation** and **discretization** — turning continuous mathematical problems into ones that a computer can solve.

In simple terms: Many math problems (integrals, differential equations, optimization) cannot be solved exactly by hand. Numerical analysis gives us *approximate* methods to solve them using a computer.

Why is Numerical Analysis Important?

- **Machine Learning:** Training neural networks requires optimization (finding the best parameters).
- **Data Compression:** Techniques like SVD reduce the size of data while keeping important information.
- **Engineering:** Simulating heat flow, structural stress, etc., all require numerical methods.
- **Science:** Modeling population growth, weather, chemical reactions uses differential equations solved numerically.

Key Areas We Will Cover

Area	What It Means
Numerical Differentiation	Approximating derivatives (slopes) of functions using discrete points
Numerical Integration	Approximating the area under a curve (definite integrals)
Optimization	Finding the minimum or maximum of a function
Linear Algebra	Working with matrices, eigenvalues, and eigenvectors
Sparse Matrix Computations	Storing & operating on large sparse matrices; LU factorization fill-in
Reinforcement Learning	Teaching agents to make optimal decisions through trial and error

Taylor Series & Numerical Differentiation

Taylor Series Expansion

Taylor Series – The Big Idea

The Taylor series lets you **approximate any smooth function** as a polynomial (sum of powers of x). If you know a function's value and all its derivatives at one point a , you can predict its value near a .

Think of it like this: If you know your position, speed, acceleration, jerk, etc. at time $t = a$, you can predict where you'll be at time $t = a + h$.

Taylor Series Formula

The Taylor series of $f(x)$ expanded around the point $x = a$ is:

$$f(x) = f(a) + f'(a)(x - a) + \frac{f''(a)}{2!}(x - a)^2 + \frac{f'''(a)}{3!}(x - a)^3 + \dots$$

Or more compactly:

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n$$

where $f^{(n)}(a)$ is the n -th derivative of f evaluated at a , and $n! = n \times (n - 1) \times \dots \times 1$.

Example: Taylor Series of e^x around $a = 0$

We know: $f(x) = e^x$, and all derivatives of e^x are e^x , so $f^{(n)}(0) = 1$ for all n .

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \dots$$

For example, $e^1 \approx 1 + 1 + 0.5 + 0.167 + 0.042 = 2.709$ (actual: 2.718...).

Numerical Differentiation

What is Numerical Differentiation?

Numerical differentiation means approximating the derivative $f'(x)$ using values of f at discrete points, rather than using calculus formulas.

Why? Sometimes we only have data points (e.g., temperature measurements every hour), not a formula. We still want to know the rate of change.

2.2.1 Forward Difference Formula

Forward Difference

$$f'(x) \approx \frac{f(x + h) - f(x)}{h}$$

Accuracy: $O(h)$ — the error is proportional to h (first order).

Intuition: This is just the slope of the line from $(x, f(x))$ to $(x + h, f(x + h))$.

2.2.2 Central Difference Formula

Central Difference – More Accurate!

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}$$

Accuracy: $O(h^2)$ — the error is proportional to h^2 (second order). This is **much better** than forward difference!

Intuition: Instead of looking only forward, we look both forward and backward — this cancels out some error terms.

Example: Approximating $f'(1)$ for $f(x) = x^2$

Exact answer: $f'(x) = 2x$, so $f'(1) = 2$.

Central difference with $h = 0.1$:

$$f'(1) \approx \frac{f(1.1) - f(0.9)}{2(0.1)} = \frac{1.21 - 0.81}{0.2} = \frac{0.40}{0.2} = 2.0 \quad (\text{Exact!})$$

Order of Accuracy – What Does $O(h^n)$ Mean?

- $O(h)$: If you halve h , the error halves. (First order)
- $O(h^2)$: If you halve h , the error reduces by a factor of 4. (Second order)
- $O(h^4)$: If you halve h , the error reduces by a factor of 16. (Fourth order)

Higher order = much more accurate for small h .

Numerical Integration

What is Numerical Integration?

Numerical integration approximates the value of a definite integral $\int_a^b f(x) dx$ (the area under the curve $f(x)$ from a to b) using numerical methods.

Why? Many integrals cannot be solved analytically (e.g., $\int_0^1 e^{-x^2} dx$). Numerical methods give us an approximate answer.

Trapezoidal Rule

Trapezoidal Rule

Divide $[a, b]$ into n equal subintervals of width $h = \frac{b-a}{n}$. Then:

$$\int_a^b f(x) dx \approx \frac{h}{2} \left[f(a) + f(b) + 2 \sum_{i=1}^{n-1} f(a + ih) \right]$$

Error: $O(h^2)$ — second order accuracy.

Intuition: We approximate the curve with straight lines (trapezoids) between consecutive points.

Simpson's Rule

Simpson's Rule – The Key Formula

Simpson's rule approximates the integral by fitting **parabolas** (degree-2 polynomials) through groups of three consecutive points:

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right]$$

For composite Simpson's rule (n subintervals, n must be even):

$$\int_a^b f(x) dx \approx \frac{h}{3} \left[f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + \cdots + 4f(x_{n-1}) + f(x_n) \right]$$

Pattern of coefficients: 1, 4, 2, 4, 2, $\dots$, 4, 2, 4, 1.

Error: $O(h^4)$ — **fourth order accuracy!** Much better than trapezoidal rule.

Simpson's Rule Error Formula

$$E = -\frac{h^4}{180} f^{(4)}(c)$$

where c is some point in $[a, b]$ and $f^{(4)}(c)$ is the fourth derivative of f at c .

Key insight: Simpson's rule is *exact* for polynomials of degree ≤ 3 (because the 4th derivative of a cubic is zero!).

Comparison of Integration Methods

Method	Order	Error	Description
Trapezoidal Rule	$O(h^2)$	$\sim h^2 f''(c)$	Connects points with straight lines
Simpson's Rule	$O(h^4)$	$\sim h^4 f^{(4)}(c)$	Fits parabolas through 3 points
Gaussian Quadrature	Very high	Depends on n	Uses optimal node positions

Gaussian Quadrature

Gaussian Quadrature – The Smart Approach

Instead of using equally-spaced points, Gaussian quadrature chooses **optimal positions** (nodes) and **weights** to get the highest possible accuracy with the fewest function evaluations.

$$\int_a^b f(x) dx \approx \sum_{i=1}^n w_i \cdot f(x_i)$$

where x_i are the **nodes** (special points) and w_i are the **weights**.

Key Properties of Gaussian Quadrature

- With n nodes, Gaussian quadrature exactly integrates polynomials of degree up to $2n - 1$.
- The nodes are roots of **Legendre polynomials** (for the standard interval $[-1, 1]$).
- **Change of variable**: To integrate over $[a, b]$, transform to $[-1, 1]$:

$$x = \frac{b-a}{2}t + \frac{a+b}{2}, \quad dx = \frac{b-a}{2}dt$$

2-Point Gaussian Quadrature

For 2 nodes on $[-1, 1]$:

- Nodes: $x_1 = -\frac{1}{\sqrt{3}}, \quad x_2 = +\frac{1}{\sqrt{3}}$
- Weights: $w_1 = w_2 = 1$

$$\int_{-1}^1 f(x) dx \approx f\left(-\frac{1}{\sqrt{3}}\right) + f\left(\frac{1}{\sqrt{3}}\right)$$

This is **exact for polynomials up to degree 3** — with only 2 function evaluations! The value $\frac{1}{\sqrt{3}} \approx 0.5774$ is one of the “magic numbers” of Gaussian quadrature.

Minimum Number of Evaluations

To exactly integrate a polynomial of degree n :

- Gaussian quadrature needs $\lceil (n+1)/2 \rceil$ nodes.
- Simpson's rule needs more points for the same accuracy.

Gaussian quadrature is the most efficient method in terms of function evaluations.

Introduction to Optimization

What is Optimization?

Optimization is the process of finding the **best solution** from a set of possible solutions. Mathematically:

$$\min_x f(x) \quad (\text{find the } x \text{ that makes } f(x) \text{ as small as possible})$$

- $f(x)$ is called the **objective function** (what we want to minimize or maximize).
- x is the **variable** (what we can change).
- There may be **constraints** (rules that x must satisfy).

Key insight: Maximizing $f(x)$ is the same as minimizing $-f(x)$. So we only need to study minimization.

Why is Optimization Important in Machine Learning?

Machine learning is **almost entirely** about optimization:

- **Training a model** = minimizing a loss function (difference between predictions and actual data).
- **Linear regression:** $\min_w \|Xw - y\|^2$
- **Neural networks:** $\min_{\theta} \sum_i L(f_{\theta}(x_i), y_i)$
- **SVM, Logistic Regression, etc.:** All formulated as optimization problems.

Types of Optimization Problems

Type	Description
Unconstrained	No restrictions on x : just minimize $f(x)$
Constrained	x must satisfy certain conditions (e.g., $x \geq 0$, $g(x) \leq 0$)
Convex	Both f and the constraint set are convex $\Rightarrow$ guaranteed global minimum
Non-convex	May have multiple local minima $\Rightarrow$ harder to solve

Standard Form of an Optimization Problem

Standard Form

$$\begin{array}{ll} \min_x & f(x) \\ \text{subject to} & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{array}$$

- $f(x)$: objective function (what we minimize)
- $g_i(x) \leq 0$: inequality constraints

- $h_j(x) = 0$: equality constraints
- **Important:** Right-hand side of all constraints must be **zero**.

Converting to Standard Form

- Maximizing $f(x) \Rightarrow$ Minimize $-f(x)$
- Constraint $g(x) \geq 0 \Rightarrow -g(x) \leq 0$
- Constraint $g(x) \leq c \Rightarrow g(x) - c \leq 0$
- Inequality $g(x) \leq 0$ can be converted to equality $g(x) + s = 0$ with $s \geq 0$ (**slack variable**)

Convex Sets & Convex Functions

Why Convexity Matters

Convexity is the **most important property** in optimization:

- If f is convex and the feasible set is convex, then **every local minimum is a global minimum**.
- This means we can find THE best solution, not just A good solution.
- Many efficient algorithms exist specifically for convex problems.

Convex Sets

Convex Set

A set C is **convex** if for *any* two points $x, y \in C$, the entire line segment between them is also in C :

$$\theta x + (1 - \theta)y \in C \quad \text{for all } \theta \in [0, 1]$$

Visual test: Pick any two points in the set. Draw a straight line between them. If the line stays inside the set, it's convex.

Examples of Convex and Non-Convex Sets

Convex: Circle, rectangle, triangle, any line segment, the entire $\mathbb{R}^n$.

Non-convex: Crescent/moon shape, star shape, the letter “L”, a donut shape.

5.1.1 Important Types of Convex Sets

Convex Set	Definition
Half-space	$\{x : a^T x \leq b\}$ — one side of a hyperplane
Polyhedron	Intersection of finitely many half-spaces: $\{x : Ax \leq b\}$
Ellipsoid	$\{x : (x - x_0)^T P^{-1}(x - x_0) \leq 1\}$ where $P \succ 0$
Convex hull	Smallest convex set containing a given set of points
Cone	$\{x : x = \alpha_1 v_1 + \alpha_2 v_2 + \dots, \alpha_i \geq 0\}$

Key Property: Intersection of Convex Sets

The **intersection** of any number of convex sets is convex. (But the union is generally *not* convex.)

This is important because the feasible set of an optimization problem is the intersection of all constraint sets.

Convex Functions

Convex Function

A function $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is **convex** if for any x, y in its domain and any $\theta \in [0, 1]$:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y)$$

Geometric meaning: The line segment connecting any two points on the graph of f lies **above or on** the graph. The function “curves upward.”

Concave Function

A function f is **concave** if $-f$ is convex. Equivalently:

$$f(\theta x + (1 - \theta)y) \geq \theta f(x) + (1 - \theta)f(y)$$

Geometric meaning: The curve bends **downward**.

Common Convex and Concave Functions

Function	Convex/Concave?	Why?
$f(x) = x^2$	Convex	$f''(x) = 2 > 0$
$f(x) = e^x$	Convex	$f''(x) = e^x > 0$
$f(x) = x $	Convex	V-shape, curves up
$f(x) = \log x$	Concave	$f''(x) = -1/x^2 < 0$
$f(x) = -x^2$	Concave	$f''(x) = -2 < 0$
$f(x) = ax + b$	Both!	Linear functions are both convex and concave

Three Ways to Check Convexity

Method 1: Definition (Zero-th Order)

Check the inequality directly:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y) \quad \forall x, y, \theta \in [0, 1]$$

This is the most basic but often hardest to verify.

Method 2: First-Order Condition

If f is differentiable, f is convex if and only if:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) \quad \text{for all } x, y$$

Meaning: The function always lies **above its tangent line/plane** at every point.

Method 3: Second-Order Condition (Most Useful!)

If f is twice differentiable, f is convex if and only if:

$$\nabla^2 f(x) \succeq 0 \quad \text{for all } x$$

For single variable: $f''(x) \geq 0$ for all $x \Rightarrow f$ is convex.

For multiple variables: The **Hessian matrix** $\nabla^2 f(x)$ must be **positive semi-definite** (PSD) for all x .

Midpoint Convexity

A continuous function is convex if and only if:

$$f\left(\frac{x+y}{2}\right) \leq \frac{f(x) + f(y)}{2} \quad \text{for all } x, y$$

Meaning: The function value at the midpoint is $\leq$ the average of the function values at the endpoints.

Linear Algebra for Optimization

Eigenvalues and Eigenvectors

Eigenvalue and Eigenvector

For a square matrix A , a non-zero vector v is an **eigenvector** if:

$$Av = \lambda v$$

where λ is the corresponding **eigenvalue**.

Intuition: When matrix A acts on eigenvector v , it only **stretches or shrinks** v (by factor λ) without changing its direction.

- $\lambda > 1$: stretches the vector
- $0 < \lambda < 1$: shrinks the vector
- $\lambda < 0$: reverses and scales the vector
- $\lambda = 0$: maps the vector to zero

Finding Eigenvalues

For matrix $A = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$:

1. Find eigenvalues by solving $\det(A - \lambda I) = 0$:

$$\det \begin{pmatrix} 2 - \lambda & 1 \\ 0 & 3 - \lambda \end{pmatrix} = (2 - \lambda)(3 - \lambda) = 0$$

2. Eigenvalues: $\lambda_1 = 2, \lambda_2 = 3$.

Positive Semi-Definite (PSD) Matrices

Positive Semi-Definite Matrix

A symmetric matrix A is **positive semi-definite** ($A \succeq 0$) if:

$$x^T A x \geq 0 \quad \text{for all vectors } x$$

Equivalent conditions (any one implies the others):

1. All eigenvalues of A are ≥ 0
2. $x^T A x \geq 0$ for all x
3. $A = B^T B$ for some matrix B

If all eigenvalues are **strictly positive** (> 0), then A is **positive definite** ($A \succ 0$).

Connection to Convexity

For a twice-differentiable function $f(x)$:

$$f \text{ is convex} \iff \text{Hessian } \nabla^2 f(x) \text{ is positive semi-definite for all } x$$

The Hessian is the matrix of all second partial derivatives:

$$[\nabla^2 f(x)]_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

Checking Convexity of $f(x, y) = x^2 + xy + y^2$

The Hessian is:

$$\nabla^2 f = \begin{pmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$$

Eigenvalues: $\lambda = 3$ and $\lambda = 1$ (both > 0). So f is **strictly convex**.

Ellipsoids and Matrices

How Matrices Create Ellipsoids

When a matrix A acts on a unit circle (sphere), it transforms it into an ellipsoid:

- The **eigenvectors** determine the **directions** of the ellipsoid's axes.
- The **eigenvalues** determine the **lengths** of the axes.

An ellipsoid centered at x_0 with shape matrix $P \succ 0$:

$$\mathcal{E} = \{x : (x - x_0)^T P^{-1} (x - x_0) \leq 1\}$$

Gradient Descent

Gradient Descent – The Most Important Algorithm

Gradient descent is an iterative algorithm to find the minimum of a function. It works by repeatedly taking steps in the direction that decreases f the fastest.

Analogy: Imagine you're on a mountain in fog and want to reach the valley. At each step, you feel the slope around you and walk in the steepest downhill direction.

Gradient Descent Update Rule

Starting from an initial guess x_0 , repeat:

$$x_{k+1} = x_k - \alpha \cdot \nabla f(x_k)$$

- x_k : current position (estimate of the minimum)
- $\nabla f(x_k)$: **gradient** of f at x_k (points in direction of steepest *increase*)
- $-\nabla f(x_k)$: **negative gradient** (points in direction of steepest *decrease*)
- $\alpha > 0$: **learning rate** / step size (how far to step)

The Learning Rate α

The learning rate is **critical**:

α too large	Overshoots the minimum, may diverge (oscillate wildly)
α too small	Converges very slowly, wastes computation
α just right	Converges quickly to the minimum

Descent Direction

What Makes a Good Descent Direction?

A direction d is a **descent direction** at point x if moving along d decreases f :

$$\nabla f(x)^T d < 0$$

The **steepest descent direction** is $d = -\nabla f(x)$ (the negative gradient).

Why? The gradient $\nabla f(x)$ points in the direction of maximum increase of f . So $-\nabla f(x)$ points in the direction of maximum decrease.

Constrained Optimization & Indicator Functions

Indicator Function

To turn a constrained problem into an unconstrained one, use an **indicator function**:

$$I_C(x) = \begin{cases} 0 & \text{if } x \in C \\ +\infty & \text{if } x \notin C \end{cases}$$

Then: $\min_{x \in C} f(x)$ becomes $\min_x [f(x) + I_C(x)]$

Problem: The indicator function is **not differentiable**, so we can't directly use gradient descent.

Subgradients

Subgradient – Handling Non-Differentiable Functions

For a convex function f that may not be differentiable everywhere, a vector g is a **subgradient** of f at x if:

$$f(y) \geq f(x) + g^T(y - x) \quad \text{for all } y$$

The set of all subgradients at x is called the **subdifferential**, denoted $\partial f(x)$.

Example: For $f(x) = |x|$ at $x = 0$: any $g \in [-1, 1]$ is a subgradient.

If f is differentiable: The subdifferential contains only one element — the gradient.

Norms

Norms – Measuring Size

A **norm** $\|\cdot\|$ assigns a non-negative “size” to each vector. It must satisfy:

1. **Positivity:** $\|x\| \geq 0$, and $\|x\| = 0 \iff x = 0$
2. **Homogeneity:** $\|\alpha x\| = |\alpha| \cdot \|x\|$
3. **Triangle inequality:** $\|x + y\| \leq \|x\| + \|y\|$

Common Norms

Norm	Formula	Meaning
L_1 norm	$\ x\ _1 = \sum_i x_i $	Sum of absolute values (“Manhattan distance”)
L_2 norm	$\ x\ _2 = \sqrt{\sum_i x_i^2}$	Euclidean distance (standard distance)
L_∞ norm	$\ x\ _\infty = \max_i x_i $	Maximum absolute value

All norms are **convex functions**.

Lagrange Multipliers & the Lagrangian

The Key Idea

Problem: How do we solve $\min f(x)$ subject to constraints?

Answer: Combine the objective and constraints into a single function — the **Lagrangian** — using **Lagrange multipliers**.

The Lagrangian Function

Lagrangian – The Master Formula

For the standard form problem:

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

The **Lagrangian** is:

$$L(x, \lambda, \mu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{j=1}^p \mu_j h_j(x)$$

- $\lambda_i \geq 0$: Lagrange multipliers for **inequality** constraints
- μ_j : Lagrange multipliers for **equality** constraints (can be any real number)

How to Form the Lagrangian – Step by Step

1. Write the problem in **standard form** (RHS of all constraints = 0).
2. Create the Lagrangian: start with $f(x)$.
3. For each inequality constraint $g_i(x) \leq 0$: add $+\lambda_i g_i(x)$.
4. For each equality constraint $h_j(x) = 0$: add $+\mu_j h_j(x)$.
5. Require $\lambda_i \geq 0$ for all i .

Example: Forming a Lagrangian

Problem: $\min x^2 + y^2$ subject to $x + y = 1$

Standard form: $h(x, y) = x + y - 1 = 0$

Lagrangian:

$$L(x, y, \mu) = x^2 + y^2 + \mu(x + y - 1)$$

Setting $\nabla_x L = 0$:

$$\begin{aligned} \frac{\partial L}{\partial x} = 2x + \mu = 0 &\implies x = -\mu/2 \\ \frac{\partial L}{\partial y} = 2y + \mu = 0 &\implies y = -\mu/2 \end{aligned}$$

From constraint: $x + y = 1 \implies -\mu = 1 \implies \mu = -1$, so $x = y = 1/2$.

Minimum: $f(1/2, 1/2) = 1/4 + 1/4 = 1/2$.

Feasible Set

Feasible Set

The **feasible set** is the set of all points x that satisfy *all* the constraints:

$$\mathcal{F} = \{x : g_i(x) \leq 0 \ \forall i, \ h_j(x) = 0 \ \forall j\}$$

The optimal solution must lie **inside** the feasible set.

Duality in Optimization

What is Duality?

Every optimization problem (the **primal** problem) has a companion problem called the **dual** problem. Duality gives us:

- A **lower bound** on the optimal value of the primal problem.
- Sometimes, an **easier** problem to solve.
- The dual is **always concave**, even if the primal is non-convex!

The Dual Function

Dual Function

The **dual function** $g(\lambda, \mu)$ is obtained by minimizing the Lagrangian over x :

$$g(\lambda, \mu) = \inf_x L(x, \lambda, \mu) = \inf_x \left[f(x) + \sum_i \lambda_i g_i(x) + \sum_j \mu_j h_j(x) \right]$$

Key properties:

1. $g(\lambda, \mu)$ is **always concave** (regardless of the primal problem's convexity).
2. $g(\lambda, \mu)$ provides a **lower bound**: $g(\lambda, \mu) \leq p^*$ (optimal primal value).
3. The Lagrangian is **affine** (linear) in λ and μ , which is why g is concave.

The Dual Problem

Dual Problem

The dual problem is to **maximize** the dual function:

$$\begin{array}{ll} \max_{\lambda, \mu} & g(\lambda, \mu) \\ \text{subject to} & \lambda_i \geq 0, \quad i = 1, \dots, m \end{array}$$

Let d^* be the optimal dual value. Then:

$$d^* \leq p^* \quad (\text{always true — this is called } \mathbf{weak\ duality})$$

Weak and Strong Duality

Weak Duality vs. Strong Duality

Weak Duality	$d^* \leq p^*$ (always holds). The dual always gives a lower bound.
Strong Duality	$d^* = p^*$ (holds under certain conditions). The dual gives the exact answer!
Duality Gap	$p^* - d^*$. If strong duality holds, the gap is zero .

Slater's Condition

Slater's Condition for Strong Duality

Strong duality holds if:

1. The primal problem is **convex** (i.e., f and all g_i are convex, and h_j are affine).
2. There exists a **strictly feasible point** $\tilde{x}$ such that:

$$g_i(\tilde{x}) < 0 \quad \text{for all } i = 1, \dots, m$$

(The inequality constraints are *strictly* satisfied, not just ≤ 0 .)

In words: If the problem is convex and there's a point that is “inside” the feasible region (not on the boundary), then strong duality holds.

Conjugate Functions

Conjugate Function (Fenchel Conjugate)

The **conjugate** of a function $f(x)$ is:

$$f^*(y) = \sup_x [y^T x - f(x)]$$

Intuition: The conjugate describes the function through its “slopes” rather than its “values.” It’s like switching from describing a curve by its points to describing it by its tangent lines.

Key properties:

- f^* is **always convex**, even if f is not!
- For convex f : $(f^*)^* = f$ (the conjugate of the conjugate gives back the original).
- **Fenchel’s inequality:** $f(x) + f^*(y) \geq x^T y$ for all x, y .

Conjugate of $f(x) = \frac{1}{2}x^2$

$$f^*(y) = \sup_x \left[xy - \frac{1}{2}x^2 \right]$$

Take derivative and set to zero: $y - x = 0 \implies x = y$. Substitute:

$$f^*(y) = y \cdot y - \frac{1}{2}y^2 = \frac{1}{2}y^2$$

So the conjugate of $\frac{1}{2}x^2$ is $\frac{1}{2}y^2$ — it’s the same function! This is called **self-conjugate**.

Connection to Duality

Conjugate Functions and Duality

The dual function can often be expressed in terms of conjugate functions. If the primal problem is:

$$\min_x f(x) \quad \text{s.t.} \quad Ax = b$$

Then the dual function involves f^* (the conjugate of f).

This connection is one of the deepest ideas in optimization: you can solve problems by looking at them from the “conjugate perspective.”

Wave-Particle Duality Analogy

Physics Analogy for Duality

In physics, **wave-particle duality** says that light and electrons can behave as both waves and particles depending on how you observe them.

Similarly, in optimization:

- The **primal** view: describes the function by its values $f(x)$.
- The **dual** view: describes the function by its slopes/tangents (conjugate).

Just as time domain $\leftrightarrow$ frequency domain via **Fourier transform**, the function $\leftrightarrow$ conjugate via the **Legendre/Fenchel transform**.

KKT Conditions

Karush-Kuhn-Tucker (KKT) Conditions

The KKT conditions are **necessary conditions for optimality** in constrained optimization. If x^* is an optimal solution and strong duality holds, then there exist λ^*, μ^* such that:

The Four KKT Conditions

1. **Stationarity:**

$$\nabla f(x^*) + \sum_{i=1}^m \lambda_i^* \nabla g_i(x^*) + \sum_{j=1}^p \mu_j^* \nabla h_j(x^*) = 0$$

(The gradient of the Lagrangian with respect to x is zero.)

2. **Primal Feasibility:**

$$g_i(x^*) \leq 0, \quad h_j(x^*) = 0$$

(The optimal point satisfies all constraints.)

3. **Dual Feasibility:**

$$\lambda_i^* \geq 0$$

(Lagrange multipliers for inequality constraints are non-negative.)

4. **Complementary Slackness:**

$$\lambda_i^* \cdot g_i(x^*) = 0 \quad \text{for all } i$$

(For each inequality constraint, either the constraint is active [$g_i(x^*) = 0$] or the multiplier is zero [$\lambda_i^* = 0$].)

Understanding Complementary Slackness

Complementary slackness means: at the optimum, for each inequality constraint, one of two things is true:

- The constraint is **active** (binding): $g_i(x^*) = 0$ — the optimal point is *on* the constraint boundary.
- The constraint is **inactive**: $g_i(x^*) < 0$ — the constraint doesn't matter, so $\lambda_i^* = 0$.

Intuition: You only “pay attention” to constraints that are actually limiting you.

When are KKT Conditions Sufficient?

For **convex** problems, the KKT conditions are both **necessary and sufficient** for optimality (when Slater's condition holds).

This means: if you find x^*, λ^*, μ^* satisfying all four KKT conditions, then x^* is **THE optimal solution**.

Applications of Optimization

Low-Rank Matrix Factorization

Low-Rank Matrix Factorization

Given a large matrix M (e.g., user-movie ratings), approximate it as:

$$M \approx UV^T$$

where U is $m \times k$ and V is $n \times k$, with $k \ll \min(m, n)$.

Why? This dramatically reduces the number of parameters and reveals hidden structure.

Recommendation Systems

How Netflix/Amazon Recommendations Work

1. **Rating Matrix:** Rows = users, Columns = movies. Entry (i, j) = user i 's rating for movie j . Most entries are **missing**.
2. **Goal:** Predict the missing entries (what rating would user i give to movie j ?).
3. **Method:** Factor the matrix $M \approx UV^T$ where:
 - Row i of U = “feature vector” of user i (e.g., likes action, dislikes romance)
 - Row j of V = “feature vector” of movie j (e.g., amount of action, romance)
 - Predicted rating = $U_i \cdot V_j$ (dot product)
4. **Optimization:** $\min_{U,V} \sum_{(i,j) \text{ observed}} (M_{ij} - U_i V_j^T)^2 + \text{regularization}$

Singular Value Decomposition (SVD)

SVD

Any matrix M can be decomposed as:

$$M = U\Sigma V^T$$

- U : orthogonal matrix (left singular vectors)
- Σ : diagonal matrix of **singular values** $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$
- V : orthogonal matrix (right singular vectors)

For compression: Keep only the top k singular values to get the best rank- k approximation.

Sparse Matrix Computations & RL Applications

From Static to Dynamic Optimization

In previous lectures we studied **static optimization**: minimize $f(x)$ subject to fixed constraints. Now we extend to **dynamic optimization**, where the **time component** matters — the decisions you make now affect future outcomes.

Key difference:

- **Static:** One-shot decision, fixed constraints.
- **Dynamic:** Sequential decisions over time; you adapt based on feedback.

Reinforcement learning is the algorithmic framework for dynamic optimization.

LU Factorization Recap

LU Decomposition

Given a matrix A , we factorize it as:

$$A = LU$$

where L is **lower triangular** and U is **upper triangular**.

Why? To solve $Ax = b$:

1. Factor $A = LU$.
2. Solve $Ly = b$ (forward substitution — easy since L is lower triangular).
3. Solve $Ux = y$ (backward substitution — easy since U is upper triangular).

The factorization works by performing row operations to eliminate entries below the diagonal column by column. These row operations correspond to multiplying by elementary lower triangular matrices.

Sparse Matrices

What is a Sparse Matrix?

A **sparse matrix** is a matrix where **most entries are zero**. In real-world applications (internet connectivity, social networks, circuit simulations), matrices can be extremely large (up to 100 million rows) but most entries are zero.

Why care? Storing all n^2 entries wastes memory. We only need to store the non-zero entries.

13.2.1 Coordinate (COO) Storage Format

Coordinate Format

Store each non-zero entry as a triplet (i, j, value) :

$$\text{Store: } \{(i_1, j_1, v_1), (i_2, j_2, v_2), \dots, (i_k, j_k, v_k)\}$$

where k is the number of non-zeros (**nnz**).

Memory: $3 \times \text{nnz}$ (three arrays: row indices, column indices, values).

13.2.2 Compressed Sparse Row (CSR) Format

CSR Format – More Efficient!

CSR reduces storage further by compressing the row information:

- **values[]**: All non-zero values (row by row).
- **col_index[]**: Column index of each non-zero.
- **row_ptr[]**: For each row, the index in **values[]** where that row starts.

Memory: $2 \times \text{nnz} + (n + 1)$, which is less than the $3 \times \text{nnz}$ of COO format.

Example: For a 4×4 matrix with non-zeros at positions $(0, 0) = 5$, $(0, 2) = 3$, $(1, 1) = 7$, $(2, 0) = 1$, $(2, 3) = 4$, $(3, 3) = 2$:

- **values** = [5, 3, 7, 1, 4, 2]
- **col_index** = [0, 2, 1, 0, 3, 3]
- **row_ptr** = [0, 2, 3, 5, 6] (row 0 starts at index 0, row 1 at 2, etc.)

Challenges with Sparse Matrix Operations

Performing operations (addition, multiplication, LU factorization) on sparse matrices in sparse format is non-trivial:

- **Addition:** Cannot simply add element-wise; must merge non-zero patterns. Output sparsity is unknown in advance.
- **Multiplication:** Result can have very different non-zero pattern. Memory allocation is tricky since you don't know the output size beforehand.
- **LU factorization:** Row operations can create new non-zeros where there were zeros before — this is called **fill-in**.

All operations must keep the result in sparse format (never “densify” the matrix).

The Fill-in Problem

Fill-in During LU Factorization

When performing LU factorization on a sparse matrix, row operations can turn zero entries into non-zeros. These new non-zeros are called **fill-in**.

Problem: Excessive fill-in destroys sparsity, increasing memory usage and computation time.

Goal: Find a **reordering** (permutation) of rows and columns such that LU factorization produces **minimum fill-in**.

Fill-in Minimization is NP-Hard!

Finding the *optimal* row/column ordering that minimizes fill-in is **NP-hard** — there is no known polynomial-time algorithm. For a 100 million $\times$ 100 million matrix, brute-force search is computationally prohibitive.

This is why we turn to **approximate methods** and **reinforcement learning** to find good (though not provably optimal) orderings.

Formulating Sparse Matrix Reordering as an RL Game

The Game Formulation

We can treat the sparse matrix fill-in minimization as a **game** where an RL agent learns to make good reordering decisions:

RL Component	Sparse Matrix Equivalent
State	Current state of the matrix (which rows/columns have been processed, current non-zero pattern)
Action	Choose which pivot row/column to process next (reordering decision)
Reward	Negative of the number of new fill-in entries created (less fill-in = higher reward)
Episode	One complete LU factorization with a particular ordering

The agent plays this “game” **many times** on many matrices, learning from experience which ordering strategies lead to less fill-in.

Why Games Are Good for RL

Games provide well-defined environments for testing RL ideas because:

- **States are precisely defined** (e.g., chess board positions, matrix non-zero patterns).
- **Actions are discrete and clear** (e.g., move a piece, choose a pivot).
- **Rewards are measurable** (win/loss, fill-in count).
- **Trial and error is cheap** (can simulate millions of games quickly).

This is why RL research started with games (chess, Go, Atari) before moving to real-world problems.

How the Agent Learns

1. The agent sees the current sparse matrix state.
2. It chooses a pivot row (action) from multiple possibilities.
3. After elimination on that row, it counts the new fill-in entries (reward signal).
4. It repeats for the next column (fewer choices remaining each time).
5. After many complete episodes, the agent builds intuition about which orderings minimize fill-in.

Key challenge: The consequence of choosing a pivot *now* can propagate far into the future — a bad early choice may cause massive fill-in later. This is the **long-term credit assignment problem**, which makes RL necessary (simple greedy approaches fail).

Results: RL Beats Classical Heuristics

The RL-based approach for sparse matrix reordering:

- **Matches or beats** all existing classical heuristics in terms of fill-in reduction.
- Achieves **better sparsity** in the L and U factors.
- Demonstrates that NP-hard problems can be **approximately solved** using RL when exact solutions are computationally infeasible.

This is one of many examples where hard combinatorial problems are formulated as games and tackled with RL.

Other RL Applications Discussed in Class

Cart-Pole Balancing Problem

A classic RL benchmark:

- **State:** Cart position, cart velocity, pole angle, pole angular velocity.
- **Action:** Apply horizontal force left or right.
- **Reward:** +1 for every time step the pole stays upright.
- **Goal:** Keep the pole balanced as long as possible.

Why not just use physics? A physics-based differential equation model is approximate and cannot account for all real-world perturbations (wind, friction variations). RL learns to handle uncertainty through **trial and error** — it builds robust intuition at the *feature level*, not precise measurements.

Grid World Example

A simple environment to understand RL concepts:

- Agent starts at some cell, must reach a goal cell (marked ★).
- **Actions:** Up, down, left, right.
- **Reward:** -1 per step (encourages reaching the goal quickly).
- A **random policy** takes many steps; an **optimal policy** finds the shortest path.

The agent learns by trying many paths, accumulating negative rewards for long paths, and gradually discovering that shorter paths yield higher cumulative reward.

Key Insight: Learning at the Feature Level

Approximate, Not Precise

A recurring theme from the lecture:

- Real-world problems have **too much uncertainty** to model precisely.
- RL agents learn at the **concept/feature level** — just like a child learns “hot things hurt” without measuring exact temperatures.
- Neural networks capture these high-level features effectively.
- The approach is **approximate but practical** — it works where exact mathematical solutions are infeasible.

This philosophy extends beyond games: modern AI coding assistants, language models, and reasoning systems all use RL (specifically RLHF — Reinforcement Learning from Human Feedback) to improve.

Reinforcement Learning

What is Reinforcement Learning (RL)?

Reinforcement Learning is a type of machine learning where an **agent** learns to make **decisions** by interacting with an **environment**:

1. Agent observes the current **state** s .
2. Agent takes an **action** a .
3. Environment gives a **reward** r and transitions to a new state s' .
4. Goal: maximize total (cumulative) reward over time.

Analogy: A dog learning tricks. It does something (action), gets a treat or scolding (reward), and learns what to do next time.

Key Concepts

Policy, Value Function, Q-Function

Concept	Notation	Meaning
Policy	$\pi(a s)$	Probability of taking action a in state s . The agent's strategy.
Value Function	$V^\pi(s)$	Expected total reward starting from state s , following policy π .
Q-Function	$Q^\pi(s, a)$	Expected total reward starting from state s , taking action a , then following π .

Relationship:

$$V^\pi(s) = \sum_a \pi(a|s) \cdot Q^\pi(s, a)$$

The value of a state = average over all actions of the Q-values, weighted by the policy.

Exploration vs. Exploitation

The Exploration-Exploitation Dilemma

Exploration	Try new, unknown actions to discover potentially better strategies.
Exploitation	Use the best known action to maximize immediate reward.

Too much exploration: Wastes time on bad actions.

Too much exploitation: May miss much better strategies.

Common strategy — ϵ -greedy: With probability ϵ , explore (random action). With probability $1 - \epsilon$, exploit (best known action).

Actor-Critic Methods

Actor-Critic Architecture

Actor	Learns the policy $\pi(a s)$ — decides <i>what to do</i> .
Critic	Learns the value function $V(s)$ or $Q(s, a)$ — evaluates <i>how good</i> the action was.

The actor proposes actions, the critic evaluates them, and the actor improves based on the critic's feedback. Both are typically implemented as **neural networks**.

Markov Decision Process (MDP)

MDP – The Mathematical Framework for RL

An MDP is defined by:

- $\mathcal{S}$: Set of **states**
- $\mathcal{A}$: Set of **actions**
- $P(s'|s, a)$: **Transition probabilities** (probability of going to s' from s after action a)
- $R(s, a)$: **Reward function** (immediate reward for taking action a in state s)
- $\gamma \in [0, 1)$: **Discount factor** (how much to value future vs. immediate rewards)

Goal: Find policy π^* that maximizes:

$$\mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right]$$

Complete Formula Sheet

All Key Formulas for Quick Reference

Taylor Series

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x-a)^n$$

Numerical Differentiation

$$\text{Forward: } f'(x) \approx \frac{f(x+h) - f(x)}{h} \quad O(h)$$

$$\text{Central: } f'(x) \approx \frac{f(x+h) - f(x-h)}{2h} \quad O(h^2)$$

Numerical Integration

$$\text{Trapezoidal: } \int_a^b f \, dx \approx \frac{h}{2} [f(a) + 2 \sum f_i + f(b)] \quad O(h^2)$$

$$\text{Simpson's: } \int_a^b f \, dx \approx \frac{h}{3} [f_0 + 4f_1 + 2f_2 + \dots + f_n] \quad O(h^4)$$

$$\text{Gaussian: } \int_a^b f \, dx \approx \sum w_i f(x_i) \quad \text{exact for deg } 2n-1$$

Simpson's Rule Error

$$E = -\frac{h^4}{180} f^{(4)}(c)$$

Optimization

$$\text{Gradient Descent: } x_{k+1} = x_k - \alpha \nabla f(x_k)$$

$$\text{Convexity: } f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta)f(y)$$

$$\text{First-order: } f(y) \geq f(x) + \nabla f(x)^T (y-x)$$

$$\text{Second-order: } \nabla^2 f(x) \succeq 0 \quad \forall x$$

Lagrangian & Duality

$$\text{Lagrangian: } L(x, \lambda, \mu) = f(x) + \sum \lambda_i g_i(x) + \sum \mu_j h_j(x)$$

$$\text{Dual function: } g(\lambda, \mu) = \inf_x L(x, \lambda, \mu)$$

$$\text{Conjugate: } f^*(y) = \sup_x [y^T x - f(x)]$$

$$\text{Weak duality: } d^* \leq p^*$$

KKT Conditions

$$\begin{aligned}\nabla_x L &= 0, & g_i(x^*) &\leq 0, & h_j(x^*) &= 0 \\ \lambda_i^* &\geq 0, & \lambda_i^* g_i(x^*) &= 0\end{aligned}$$

Eigenvalues

$$Av = \lambda v, \quad \det(A - \lambda I) = 0$$

Norms

$$\|x\|_1 = \sum |x_i|, \quad \|x\|_2 = \sqrt{\sum x_i^2}, \quad \|x\|_\infty = \max |x_i|$$

Sparse Matrices

COO: (i, j, v) triplets CSR: `values[], col_idx[], row_ptr[]`

Fill-in minimization in LU: NP-hard $\Rightarrow$ use RL-based approximate reordering

Reinforcement Learning

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a), \quad \text{Goal: } \max_\pi \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R_t \right]$$

Practice Exam Questions with Answers

Question 1: Numerical Integration

Q: Use Simpson's Rule with $h = 0.5$ to approximate $\int_0^1 e^x dx$.

A: With $h = 0.5$: $x_0 = 0$, $x_1 = 0.5$, $x_2 = 1$.

$$\begin{aligned}\int_0^1 e^x dx &\approx \frac{0.5}{3}[e^0 + 4e^{0.5} + e^1] = \frac{0.5}{3}[1 + 4(1.6487) + 2.7183] \\ &= \frac{0.5}{3}[1 + 6.5948 + 2.7183] = \frac{0.5}{3}(10.3131) = 1.7189\end{aligned}$$

Exact: $e^1 - e^0 = 1.7183$. Error ≈ 0.0006 (very small!).

Question 2: Convexity

Q: Is $f(x) = x^4 - 2x^2$ convex on all of $\mathbb{R}$?

A: Check: $f''(x) = 12x^2 - 4$. At $x = 0$: $f''(0) = -4 < 0$.

So f is **not convex** on all of $\mathbb{R}$ (the Hessian is negative at $x = 0$).

However, for $|x| > 1/\sqrt{3}$, $f''(x) > 0$, so f is convex in those regions.

Question 3: Lagrangian

Q: Write the Lagrangian for: $\min x^2 + y^2$ subject to $x + y \leq 1$ and $x \geq 0$.

A: Standard form: $g_1(x, y) = x + y - 1 \leq 0$, $g_2(x, y) = -x \leq 0$.

$$L(x, y, \lambda_1, \lambda_2) = x^2 + y^2 + \lambda_1(x + y - 1) + \lambda_2(-x)$$

with $\lambda_1, \lambda_2 \geq 0$.

Question 4: KKT Conditions

Q: State the four KKT conditions and explain complementary slackness.

A: The four conditions are:

1. **Stationarity:** $\nabla_x L(x^*, \lambda^*, \mu^*) = 0$
2. **Primal feasibility:** $g_i(x^*) \leq 0$, $h_j(x^*) = 0$
3. **Dual feasibility:** $\lambda_i^* \geq 0$
4. **Complementary slackness:** $\lambda_i^* g_i(x^*) = 0$ for all i

Complementary slackness means: for each inequality constraint, either the constraint is *active* ($g_i = 0$) or the multiplier is zero. You don't "pay" for constraints that aren't binding.

Question 5: Sparse Matrices & RL

Q: A 5×5 sparse matrix has 8 non-zero entries. Compare the storage cost of dense format, COO format, and CSR format. Why is fill-in minimization during LU factorization formulated as an RL problem?

A:

- **Dense:** $5 \times 5 = 25$ entries stored.
- **COO:** $3 \times 8 = 24$ values (row, col, value for each non-zero).
- **CSR:** $2 \times 8 + (5 + 1) = 22$ values (values + col indices + row pointers). Most compact.

Fill-in as RL: Finding the optimal pivot ordering to minimize fill-in is NP-hard. RL is used because:

- The choice of pivot *now* affects fill-in far into the future (long-term credit assignment).
- The agent learns from many trial-and-error episodes which orderings produce less fill-in.
- RL-based approaches match or beat classical heuristics in practice.

Question 6: Conjugate Function

Q: Find the conjugate of $f(x) = |x|$.

A: $f^*(y) = \sup_x [xy - |x|]$.

If $|y| \leq 1$: the supremum is 0 (achieved at $x = 0$).

If $|y| > 1$: the expression $xy - |x|$ is unbounded (take $x \rightarrow +\infty$ if $y > 1$), so $f^*(y) = +\infty$.

Therefore:

$$f^*(y) = \begin{cases} 0 & \text{if } |y| \leq 1 \\ +\infty & \text{otherwise} \end{cases} = I_{[-1,1]}(y)$$

This is the **indicator function** of the interval $[-1, 1]$.

Question 7: Gradient Descent

Q: Perform 2 iterations of gradient descent on $f(x) = x^2$ starting at $x_0 = 4$ with $\alpha = 0.3$.

A: $f'(x) = 2x$.

Iteration 1: $x_1 = x_0 - \alpha f'(x_0) = 4 - 0.3(8) = 4 - 2.4 = 1.6$

Iteration 2: $x_2 = x_1 - \alpha f'(x_1) = 1.6 - 0.3(3.2) = 1.6 - 0.96 = 0.64$

The iterates are converging toward $x^* = 0$ (the minimum).

Question 8: Eigenvalues & Positive Definiteness

Q: Is $A = \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix}$ positive semi-definite?

A: Find eigenvalues: $\det(A - \lambda I) = (4 - \lambda)(1 - \lambda) - 4 = \lambda^2 - 5\lambda = \lambda(\lambda - 5)$.

Eigenvalues: $\lambda_1 = 0$, $\lambda_2 = 5$. Both ≥ 0 , so A is **positive semi-definite** (but not positive definite since $\lambda_1 = 0$).

Question 9: Duality

Q: What is weak duality? What additional condition ensures strong duality?

A: Weak duality: $d^* \leq p^*$ (dual optimal value $\leq$ primal optimal value). This *always* holds.

Strong duality: $d^* = p^*$. Holds when:

- The primal is convex, AND
- Slater's condition is satisfied (there exists a strictly feasible point where all inequality constraints are *strictly* satisfied: $g_i(\tilde{x}) < 0$).

Question 10: Reinforcement Learning

Q: Explain the difference between a policy and a value function.

A:

- **Policy** $\pi(a|s)$: Tells the agent *what to do* — maps each state to an action (or probability distribution over actions). It's the agent's strategy.
- **Value function** $V^\pi(s)$: Tells the agent *how good a state is* — estimates the expected total future reward from that state onward, following policy π .

The policy decides actions; the value function evaluates states. Together they enable learning optimal behavior.

Quick Revision Checklist

Things You MUST Know for the Exam

- 1 Taylor series formula and how to use it
- 2 Forward and central difference formulas + their order of accuracy
- 3 Trapezoidal rule and Simpson's rule formulas + error orders
- 4 Gaussian quadrature concept: nodes, weights, change of variable
- 5 Definition of convex set and convex function (with the inequality)
- 6 Three ways to check convexity (definition, 1st order, 2nd order / Hessian PSD)
- 7 Gradient descent update rule and the role of learning rate
- 8 Eigenvalue equation $Av = \lambda v$ and how to find eigenvalues
- 9 Positive semi-definite: definition and connection to convexity
- 10 Standard form of optimization problem
- 11 Lagrangian function: how to write it for given constraints
- 12 Dual function: definition and why it's always concave
- 13 Weak duality ($d^* \leq p^*$) and strong duality ($d^* = p^*$)
- 14 Slater's condition for strong duality
- 15 KKT conditions (all four, especially complementary slackness)
- 16 Conjugate function definition: $f^*(y) = \sup_x [y^T x - f(x)]$
- 17 Indicator function, subgradient, and norms (L_1, L_2, L_∞)
- 18 Sparse matrix storage: COO format vs. CSR format and their trade-offs
- 19 Fill-in problem in LU factorization: why it is NP-hard, how RL helps
- 20 Reinforcement learning basics: policy, value function, Q-function, exploration vs. exploitation

Good luck on your exam!

You've got this — just review the formulas and practice the problems.